I have the following data of chess rating:

pair_num player_name total_pts state rating_pre round result result_type opponent opponent_rating

1 GARY HUA 6.0 ON 1794 round1 W 39 W 39 1436

1 GARY HUA 6.0 ON 1794 round2 W 21 W 21 1563

1 GARY HUA 6.0 ON 1794 round3 W 18 W 18 1600

1 GARY HUA 6.0 ON 1794 round4 W 14 W 14 1610

1 GARY HUA 6.0 ON 1794 round5 W 7 W 7 1649

1 GARY HUA 6.0 ON 1794 round6 D 12 D 12 1663

1 GARY HUA 6.0 ON 1794 round7 D 4 D 4 1716

2 DAKSHESH DARURI 6.0 MI 1553 round1 W 63 W 63 1175

2 DAKSHESH DARURI 6.0 MI 1553 round2 W 58 W 58 917

2 DAKSHESH DARURI 6.0 MI 1553 round3 L 4 L 4 1716

2 DAKSHESH DARURI 6.0 MI 1553 round4 W 17 W 17 1629

2 DAKSHESH DARURI 6.0 MI 1553 round5 W 16 W 16 1604

2 DAKSHESH DARURI 6.0 MI 1553 round6 W 20 W 20 1595

2 DAKSHESH DARURI 6.0 MI 1553 round7 W 7 W 7 1649
3 ADITYA BAJAJ 6.0 MI 1384 round1 L 8 L 8 1641
3 ADITYA BAJAJ 6.0 MI 1384 round2 W 61 W 61 955
3 ADITYA BAJAJ 6.0 MI 1384 round3 W 25 W 25 1745
3 ADITYA BAJAJ 6.0 MI 1384 round4 W 21 W 21 1563
3 ADITYA BAJAJ 6.0 MI 1384 round5 W 11 W 11 1712
3 ADITYA BAJAJ 6.0 MI 1384 round6 W 13 W 13 1666
3 ADITYA BAJAJ 6.0 MI 1384 round7 W 12 W 12 1663
4 PATRICK H SCHILLING 5.5 MI 1716 round1 W 23 W 23 1363
4 PATRICK H SCHILLING 5.5 MI 1716 round2 D 28 D 28 1507
4 PATRICK H SCHILLING 5.5 MI 1716 round3 W 2 W 2 1553
4 PATRICK H SCHILLING 5.5 MI 1716 round4 W 26 W 26 1579
4 PATRICK H SCHILLING 5.5 MI 1716 round5 D 5 D 5 1655
4 PATRICK H SCHILLING 5.5 MI 1716 round6 W 19 W 19 1564
4 PATRICK H SCHILLING 5.5 MI 1716 round7 D 1 D 1 1794

5 HANSHI ZUO 5.5 MI 1655 round1 W 45 W 45 1242
5 HANSHI ZUO 5.5 MI 1655 round2 W 37 W 37 980
5 HANSHI ZUO 5.5 MI 1655 round3 D 12 D 12 1663
5 HANSHI ZUO 5.5 MI 1655 round4 D 13 D 13 1666
5 HANSHI ZUO 5.5 MI 1655 round5 D 4 D 4 1716
5 HANSHI ZUO 5.5 MI 1655 round6 W 14 W 14 1610
5 HANSHI ZUO 5.5 MI 1655 round7 W 17 W 17 1629
6 HANSEN SONG 5.0 OH 1686 round1 W 34 W 34 1399
6 HANSEN SONG 5.0 OH 1686 round2 D 29 D 29 1602
6 HANSEN SONG 5.0 OH 1686 round3 L 11 L 11 1712
6 HANSEN SONG 5.0 OH 1686 round4 W 35 W 35 1438
6 HANSEN SONG 5.0 OH 1686 round5 D 10 D 10 1365
6 HANSEN SONG 5.0 OH 1686 round6 W 27 W 27 1552
6 HANSEN SONG 5.0 OH 1686 round7 W 21 W 21 1563
7 GARY DEE SWATHELL 5.0 MI 1649 round1 W 57 W 57 1092

7 GARY DEE SWATHELL 5.0 MI 1649 round2 W 46 W 46 377
7 GARY DEE SWATHELL 5.0 MI 1649 round3 W 13 W 13 1666
7 GARY DEE SWATHELL 5.0 MI 1649 round4 W 11 W 11 1712
7 GARY DEE SWATHELL 5.0 MI 1649 round5 L 1 L 1 1794
7 GARY DEE SWATHELL 5.0 MI 1649 round6 W 9 W 9 1411
7 GARY DEE SWATHELL 5.0 MI 1649 round7 L 2 L 2 1553
8 EZEKIEL HOUGHTON 5.0 MI 1641 round1 W 3 W 3 1384
8 EZEKIEL HOUGHTON 5.0 MI 1641 round2 W 32 W 32 1441
8 EZEKIEL HOUGHTON 5.0 MI 1641 round3 L 14 L 14 1610
8 EZEKIEL HOUGHTON 5.0 MI 1641 round4 L 9 L 9 1411
8 EZEKIEL HOUGHTON 5.0 MI 1641 round5 W 47 W 47 1362
8 EZEKIEL HOUGHTON 5.0 MI 1641 round6 W 28 W 28 1507
8 EZEKIEL HOUGHTON 5.0 MI 1641 round7 W 19 W 19 1564
9 STEFANO LEE 5.0 ON 1411 round1 W 25 W 25 1745
9 STEFANO LEE 5.0 ON 1411 round2 L 18 L 18 1600

9 STEFANO LEE 5.0 ON 1411 round3 W 59 W 59
853
9 STEFANO LEE 5.0 ON 1411 round4 W 8 W 8
1641
9 STEFANO LEE 5.0 ON 1411 round5 W 26 W 26
1579
9 STEFANO LEE 5.0 ON 1411 round6 L 7 L 7
1649
9 STEFANO LEE 5.0 ON 1411 round7 W 20 W 20
1595
10 ANVIT RAO 5.0 MI 1365 round1 D 16 D 16
1604
10 ANVIT RAO 5.0 MI 1365 round2 L 19 L 19
1564
10 ANVIT RAO 5.0 MI 1365 round3 W 55 W 55
1186
10 ANVIT RAO 5.0 MI 1365 round4 W 31 W 31
1494
10 ANVIT RAO 5.0 MI 1365 round5 D 6 D 6
1686
10 ANVIT RAO 5.0 MI 1365 round6 W 25 W 25
1745
10 ANVIT RAO 5.0 MI 1365 round7 W 18 W 18
1600
11 CAMERON WILLIAM MC LEMAN 4.5 MI 1712
round1 D 38 D 38 1423
11 CAMERON WILLIAM MC LEMAN 4.5 MI 1712
round2 W 56 W 56 1153
11 CAMERON WILLIAM MC LEMAN 4.5 MI 1712
round3 W 6 W 6 1686

11 CAMERON WILLIAM MC LEMAN 4.5 MI 1712 round4 L 7 L 7 1649
11 CAMERON WILLIAM MC LEMAN 4.5 MI 1712 round5 L 3 L 3 1384
11 CAMERON WILLIAM MC LEMAN 4.5 MI 1712 round6 W 34 W 34 1399
11 CAMERON WILLIAM MC LEMAN 4.5 MI 1712 round7 W 26 W 26 1579
12 KENNETH J TACK 4.5 MI 1663 round1 W 42 W 42 1332
12 KENNETH J TACK 4.5 MI 1663 round2 W 33 W 33 1449
12 KENNETH J TACK 4.5 MI 1663 round3 D 5 D 5 1655
12 KENNETH J TACK 4.5 MI 1663 round4 W 38 W 38 1423
12 KENNETH J TACK 4.5 MI 1663 round6 D 1 D 1 1794
12 KENNETH J TACK 4.5 MI 1663 round7 L 3 L 3 1384
13 TORRANCE HENRY JR 4.5 MI 1666 round1 W 36 W 36 1355
13 TORRANCE HENRY JR 4.5 MI 1666 round2 W 27 W 27 1552
13 TORRANCE HENRY JR 4.5 MI 1666 round3 L 7 L 7 1649
13 TORRANCE HENRY JR 4.5 MI 1666 round4 D 5 D 5 1655
13 TORRANCE HENRY JR 4.5 MI 1666 round5 W 33 W 33 1449

13 TORRANCE HENRY JR 4.5 MI 1666 round6 L 3 L 3 1384
13 TORRANCE HENRY JR 4.5 MI 1666 round7 W 32 W 32 1441
14 BRADLEY SHAW 4.5 MI 1610 round1 W 54 W 54 1270
14 BRADLEY SHAW 4.5 MI 1610 round2 W 44 W 44 1199
14 BRADLEY SHAW 4.5 MI 1610 round3 W 8 W 8 1641
14 BRADLEY SHAW 4.5 MI 1610 round4 L 1 L 1 1794
14 BRADLEY SHAW 4.5 MI 1610 round5 D 27 D 27 1552
14 BRADLEY SHAW 4.5 MI 1610 round6 L 5 L 5 1655
14 BRADLEY SHAW 4.5 MI 1610 round7 W 31 W 31 1494
15 ZACHARY JAMES HOUGHTON 4.5 MI 1220 round1 D 19 D 19 1564
15 ZACHARY JAMES HOUGHTON 4.5 MI 1220 round2 L 16 L 16 1604
15 ZACHARY JAMES HOUGHTON 4.5 MI 1220 round3 W 30 W 30 1522
15 ZACHARY JAMES HOUGHTON 4.5 MI 1220 round4 L 22 L 22 1555
15 ZACHARY JAMES HOUGHTON 4.5 MI 1220 round5 W 54 W 54 1270
15 ZACHARY JAMES HOUGHTON 4.5 MI 1220 round6 W 33 W 33 1449

15 ZACHARY JAMES HOUGHTON 4.5 MI 1220 round7 W 38 W 38 1423
16 MIKE NIKITIN 4.0 MI 1604 round1 D 10 D 10 1365
16 MIKE NIKITIN 4.0 MI 1604 round2 W 15 W 15 1220
16 MIKE NIKITIN 4.0 MI 1604 round4 W 39 W 39 1436
16 MIKE NIKITIN 4.0 MI 1604 round5 L 2 L 2 1553
16 MIKE NIKITIN 4.0 MI 1604 round6 W 36 W 36 1355
17 RONALD GRZEGORCZYK 4.0 MI 1629 round1 W 48 W 48 1382
17 RONALD GRZEGORCZYK 4.0 MI 1629 round2 W 41 W 41 1403
17 RONALD GRZEGORCZYK 4.0 MI 1629 round3 L 26 L 26 1579
17 RONALD GRZEGORCZYK 4.0 MI 1629 round4 L 2 L 2 1553
17 RONALD GRZEGORCZYK 4.0 MI 1629 round5 W 23 W 23 1363
17 RONALD GRZEGORCZYK 4.0 MI 1629 round6 W 22 W 22 1555
17 RONALD GRZEGORCZYK 4.0 MI 1629 round7 L 5 L 5 1655
18 DAVID SUNDEEN 4.0 MI 1600 round1 W 47 W 47 1362
18 DAVID SUNDEEN 4.0 MI 1600 round2 W 9 W 9 1411

18 DAVID SUNDEEN 4.0 MI 1600 round3 L 1 L 1 1794
18 DAVID SUNDEEN 4.0 MI 1600 round4 W 32 W 32 1441
18 DAVID SUNDEEN 4.0 MI 1600 round5 L 19 L 19 1564
18 DAVID SUNDEEN 4.0 MI 1600 round6 W 38 W 38 1423
18 DAVID SUNDEEN 4.0 MI 1600 round7 L 10 L 10 1365
19 DIPANKAR ROY 4.0 MI 1564 round1 D 15 D 15 1220
19 DIPANKAR ROY 4.0 MI 1564 round2 W 10 W 10 1365
19 DIPANKAR ROY 4.0 MI 1564 round3 W 52 W 52 935
19 DIPANKAR ROY 4.0 MI 1564 round4 D 28 D 28 1507
19 DIPANKAR ROY 4.0 MI 1564 round5 W 18 W 18 1600
19 DIPANKAR ROY 4.0 MI 1564 round6 L 4 L 4 1716
19 DIPANKAR ROY 4.0 MI 1564 round7 L 8 L 8 1641
20 JASON ZHENG 4.0 MI 1595 round1 L 40 L 40 1348
20 JASON ZHENG 4.0 MI 1595 round2 W 49 W 49 1291
20 JASON ZHENG 4.0 MI 1595 round3 W 23 W 23 1363

20 JASON ZHENG 4.0 MI 1595 round4 W 41 W 41 1403
20 JASON ZHENG 4.0 MI 1595 round5 W 28 W 28 1507
20 JASON ZHENG 4.0 MI 1595 round6 L 2 L 2 1553
20 JASON ZHENG 4.0 MI 1595 round7 L 9 L 9 1411
21 DINH DANG BUI 4.0 ON 1563 round1 W 43 W 43 1283
21 DINH DANG BUI 4.0 ON 1563 round2 L 1 L 1 1794
21 DINH DANG BUI 4.0 ON 1563 round3 W 47 W 47 1362
21 DINH DANG BUI 4.0 ON 1563 round4 L 3 L 3 1384
21 DINH DANG BUI 4.0 ON 1563 round5 W 40 W 40 1348
21 DINH DANG BUI 4.0 ON 1563 round6 W 39 W 39 1436
21 DINH DANG BUI 4.0 ON 1563 round7 L 6 L 6 1686
22 EUGENE L MCCLURE 4.0 MI 1555 round1 W 64 W 64 1163
22 EUGENE L MCCLURE 4.0 MI 1555 round2 D 52 D 52 935
22 EUGENE L MCCLURE 4.0 MI 1555 round3 L 28 L 28 1507
22 EUGENE L MCCLURE 4.0 MI 1555 round4 W 15 W 15 1220

22 EUGENE L MCCLURE 4.0 MI 1555 round6 L 17 L 17 1629
22 EUGENE L MCCLURE 4.0 MI 1555 round7 W 40 W 40 1348
23 ALAN BUI 4.0 ON 1363 round1 L 4 L 4 1716
23 ALAN BUI 4.0 ON 1363 round2 W 43 W 43 1283
23 ALAN BUI 4.0 ON 1363 round3 L 20 L 20 1595
23 ALAN BUI 4.0 ON 1363 round4 W 58 W 58 917
23 ALAN BUI 4.0 ON 1363 round5 L 17 L 17 1629
23 ALAN BUI 4.0 ON 1363 round6 W 37 W 37 980
23 ALAN BUI 4.0 ON 1363 round7 W 46 W 46 377
24 MICHAEL R ALDRICH 4.0 MI 1229 round1 L 28 L 28 1507
24 MICHAEL R ALDRICH 4.0 MI 1229 round2 L 47 L 47 1362
24 MICHAEL R ALDRICH 4.0 MI 1229 round3 W 43 W 43 1283
24 MICHAEL R ALDRICH 4.0 MI 1229 round4 L 25 L 25 1745
24 MICHAEL R ALDRICH 4.0 MI 1229 round5 W 60 W 60 967
24 MICHAEL R ALDRICH 4.0 MI 1229 round6 W 44 W 44 1199

24 MICHAEL R ALDRICH 4.0 MI 1229 round7 W 39 W 39 1436
25 LOREN SCHWIEBERT 3.5 MI 1745 round1 L 9 L 9 1411
25 LOREN SCHWIEBERT 3.5 MI 1745 round2 W 53 W 53 1393
25 LOREN SCHWIEBERT 3.5 MI 1745 round3 L 3 L 3 1384
25 LOREN SCHWIEBERT 3.5 MI 1745 round4 W 24 W 24 1229
25 LOREN SCHWIEBERT 3.5 MI 1745 round5 D 34 D 34 1399
25 LOREN SCHWIEBERT 3.5 MI 1745 round6 L 10 L 10 1365
25 LOREN SCHWIEBERT 3.5 MI 1745 round7 W 47 W 47 1362
26 MAX ZHU 3.5 ON 1579 round1 W 49 W 49 1291
26 MAX ZHU 3.5 ON 1579 round2 W 40 W 40 1348
26 MAX ZHU 3.5 ON 1579 round3 W 17 W 17 1629
26 MAX ZHU 3.5 ON 1579 round4 L 4 L 4 1716
26 MAX ZHU 3.5 ON 1579 round5 L 9 L 9 1411
26 MAX ZHU 3.5 ON 1579 round6 D 32 D 32 1441
26 MAX ZHU 3.5 ON 1579 round7 L 11 L 11 1712

27 GAURAV GIDWANI 3.5 MI 1552 round1 W 51 W 51 1011
27 GAURAV GIDWANI 3.5 MI 1552 round2 L 13 L 13 1666
27 GAURAV GIDWANI 3.5 MI 1552 round3 W 46 W 46 377
27 GAURAV GIDWANI 3.5 MI 1552 round4 W 37 W 37 980
27 GAURAV GIDWANI 3.5 MI 1552 round5 D 14 D 14 1610
27 GAURAV GIDWANI 3.5 MI 1552 round6 L 6 L 6 1686
28 SOFIA ADINA STANESCU-BELLU 3.5 MI 1507 round1 W 24 W 24 1229
28 SOFIA ADINA STANESCU-BELLU 3.5 MI 1507 round2 D 4 D 4 1716
28 SOFIA ADINA STANESCU-BELLU 3.5 MI 1507 round3 W 22 W 22 1555
28 SOFIA ADINA STANESCU-BELLU 3.5 MI 1507 round4 D 19 D 19 1564
28 SOFIA ADINA STANESCU-BELLU 3.5 MI 1507 round5 L 20 L 20 1595
28 SOFIA ADINA STANESCU-BELLU 3.5 MI 1507 round6 L 8 L 8 1641
28 SOFIA ADINA STANESCU-BELLU 3.5 MI 1507 round7 D 36 D 36 1355
29 CHIEDOZIE OKORIE 3.5 MI 1602 round1 W 50 W 50 1056
29 CHIEDOZIE OKORIE 3.5 MI 1602 round2 D 6 D 6 1686

29 CHIEDOZIE OKORIE 3.5 MI 1602 round3 L 38 L 38 1423
29 CHIEDOZIE OKORIE 3.5 MI 1602 round4 L 34 L 34 1399
29 CHIEDOZIE OKORIE 3.5 MI 1602 round5 W 52 W 52 935
29 CHIEDOZIE OKORIE 3.5 MI 1602 round6 W 48 W 48 1382
30 GEORGE AVERY JONES 3.5 ON 1522 round1 L 52 L 52 935
30 GEORGE AVERY JONES 3.5 ON 1522 round2 D 64 D 64 1163
30 GEORGE AVERY JONES 3.5 ON 1522 round3 L 15 L 15 1220
30 GEORGE AVERY JONES 3.5 ON 1522 round4 W 55 W 55 1186
30 GEORGE AVERY JONES 3.5 ON 1522 round5 L 31 L 31 1494
30 GEORGE AVERY JONES 3.5 ON 1522 round6 W 61 W 61 955
30 GEORGE AVERY JONES 3.5 ON 1522 round7 W 50 W 50 1056
31 RISHI SHETTY 3.5 MI 1494 round1 L 58 L 58 917
31 RISHI SHETTY 3.5 MI 1494 round2 D 55 D 55 1186
31 RISHI SHETTY 3.5 MI 1494 round3 W 64 W 64 1163
31 RISHI SHETTY 3.5 MI 1494 round4 L 10 L 10 1365

31 RISHI SHETTY 3.5 MI 1494 round5 W 30 W 30 1522
31 RISHI SHETTY 3.5 MI 1494 round6 W 50 W 50 1056
31 RISHI SHETTY 3.5 MI 1494 round7 L 14 L 14 1610
32 JOSHUA PHILIP MATHEWS 3.5 ON 1441 round1 W 61 W 61 955
32 JOSHUA PHILIP MATHEWS 3.5 ON 1441 round2 L 8 L 8 1641
32 JOSHUA PHILIP MATHEWS 3.5 ON 1441 round3 W 44 W 44 1199
32 JOSHUA PHILIP MATHEWS 3.5 ON 1441 round4 L 18 L 18 1600
32 JOSHUA PHILIP MATHEWS 3.5 ON 1441 round5 W 51 W 51 1011
32 JOSHUA PHILIP MATHEWS 3.5 ON 1441 round6 D 26 D 26 1579
32 JOSHUA PHILIP MATHEWS 3.5 ON 1441 round7 L 13 L 13 1666
33 JADE GE 3.5 MI 1449 round1 W 60 W 60 967
33 JADE GE 3.5 MI 1449 round2 L 12 L 12 1663
33 JADE GE 3.5 MI 1449 round3 W 50 W 50 1056
33 JADE GE 3.5 MI 1449 round4 D 36 D 36 1355
33 JADE GE 3.5 MI 1449 round5 L 13 L 13 1666

33 JADE GE 3.5 MI 1449 round6 L 15 L 15 1220
33 JADE GE 3.5 MI 1449 round7 W 51 W 51 1011
34 MICHAEL JEFFERY THOMAS 3.5 MI 1399 round1 L 6 L 6 1686
34 MICHAEL JEFFERY THOMAS 3.5 MI 1399 round2 W 60 W 60 967
34 MICHAEL JEFFERY THOMAS 3.5 MI 1399 round3 L 37 L 37 980
34 MICHAEL JEFFERY THOMAS 3.5 MI 1399 round4 W 29 W 29 1602
34 MICHAEL JEFFERY THOMAS 3.5 MI 1399 round5 D 25 D 25 1745
34 MICHAEL JEFFERY THOMAS 3.5 MI 1399 round6 L 11 L 11 1712
34 MICHAEL JEFFERY THOMAS 3.5 MI 1399 round7 W 52 W 52 935
35 JOSHUA DAVID LEE 3.5 MI 1438 round1 L 46 L 46 377
35 JOSHUA DAVID LEE 3.5 MI 1438 round2 L 38 L 38 1423
35 JOSHUA DAVID LEE 3.5 MI 1438 round3 W 56 W 56 1153
35 JOSHUA DAVID LEE 3.5 MI 1438 round4 L 6 L 6 1686
35 JOSHUA DAVID LEE 3.5 MI 1438 round5 W 57 W 57 1092
35 JOSHUA DAVID LEE 3.5 MI 1438 round6 D 52 D 52 935

35 JOSHUA DAVID LEE 3.5 MI 1438 round7 W 48 W 48 1382
36 SIDDHARTH JHA 3.5 MI 1355 round1 L 13 L 13 1666
36 SIDDHARTH JHA 3.5 MI 1355 round2 W 57 W 57 1092
36 SIDDHARTH JHA 3.5 MI 1355 round3 W 51 W 51 1011
36 SIDDHARTH JHA 3.5 MI 1355 round4 D 33 D 33 1449
36 SIDDHARTH JHA 3.5 MI 1355 round6 L 16 L 16 1604
36 SIDDHARTH JHA 3.5 MI 1355 round7 D 28 D 28 1507
37 AMIYATOSH PWNANANDAM 3.5 MI 980 round2 L 5 L 5 1655
37 AMIYATOSH PWNANANDAM 3.5 MI 980 round3 W 34 W 34 1399
37 AMIYATOSH PWNANANDAM 3.5 MI 980 round4 L 27 L 27 1552
37 AMIYATOSH PWNANANDAM 3.5 MI 980 round6 L 23 L 23 1363
37 AMIYATOSH PWNANANDAM 3.5 MI 980 round7 W 61 W 61 955
38 BRIAN LIU 3.0 MI 1423 round1 D 11 D 11 1712
38 BRIAN LIU 3.0 MI 1423 round2 W 35 W 35 1438
38 BRIAN LIU 3.0 MI 1423 round3 W 29 W 29 1602

38 BRIAN LIU 3.0 MI 1423 round4 L 12 L 12
1663
38 BRIAN LIU 3.0 MI 1423 round6 L 18 L 18
1600
38 BRIAN LIU 3.0 MI 1423 round7 L 15 L 15
1220
39 JOEL R HENDON 3.0 MI 1436 round1 L 1 L 1
1794
39 JOEL R HENDON 3.0 MI 1436 round2 W 54
W 54 1270
39 JOEL R HENDON 3.0 MI 1436 round3 W 40
W 40 1348
39 JOEL R HENDON 3.0 MI 1436 round4 L 16 L
16 1604
39 JOEL R HENDON 3.0 MI 1436 round5 W 44
W 44 1199
39 JOEL R HENDON 3.0 MI 1436 round6 L 21 L
21 1563
39 JOEL R HENDON 3.0 MI 1436 round7 L 24 L
24 1229
40 FOREST ZHANG 3.0 MI 1348 round1 W 20
W 20 1595
40 FOREST ZHANG 3.0 MI 1348 round2 L 26 L
26 1579
40 FOREST ZHANG 3.0 MI 1348 round3 L 39 L
39 1436
40 FOREST ZHANG 3.0 MI 1348 round4 W 59
W 59 853
40 FOREST ZHANG 3.0 MI 1348 round5 L 21 L
21 1563

40 FOREST ZHANG 3.0 MI 1348 round6 W 56 W 56 1153
40 FOREST ZHANG 3.0 MI 1348 round7 L 22 L 22 1555
41 KYLE WILLIAM MURPHY 3.0 MI 1403 round1 W 59 W 59 853
41 KYLE WILLIAM MURPHY 3.0 MI 1403 round2 L 17 L 17 1629
41 KYLE WILLIAM MURPHY 3.0 MI 1403 round3 W 58 W 58 917
41 KYLE WILLIAM MURPHY 3.0 MI 1403 round4 L 20 L 20 1595
42 JARED GE 3.0 MI 1332 round1 L 12 L 12 1663
42 JARED GE 3.0 MI 1332 round2 L 50 L 50 1056
42 JARED GE 3.0 MI 1332 round3 L 57 L 57 1092
42 JARED GE 3.0 MI 1332 round4 D 60 D 60 967
42 JARED GE 3.0 MI 1332 round5 D 61 D 61 955
42 JARED GE 3.0 MI 1332 round6 W 64 W 64 1163
42 JARED GE 3.0 MI 1332 round7 W 56 W 56 1153
43 ROBERT GLEN VASEY 3.0 MI 1283 round1 L 21 L 21 1563
43 ROBERT GLEN VASEY 3.0 MI 1283 round2 L 23 L 23 1363

43 ROBERT GLEN VASEY 3.0 MI 1283 round3 L 24 L 24 1229
43 ROBERT GLEN VASEY 3.0 MI 1283 round4 W 63 W 63 1175
43 ROBERT GLEN VASEY 3.0 MI 1283 round5 W 59 W 59 853
43 ROBERT GLEN VASEY 3.0 MI 1283 round6 L 46 L 46 377
43 ROBERT GLEN VASEY 3.0 MI 1283 round7 W 55 W 55 1186
44 JUSTIN D SCHILLING 3.0 MI 1199 round2 L 14 L 14 1610
44 JUSTIN D SCHILLING 3.0 MI 1199 round3 L 32 L 32 1441
44 JUSTIN D SCHILLING 3.0 MI 1199 round4 W 53 W 53 1393
44 JUSTIN D SCHILLING 3.0 MI 1199 round5 L 39 L 39 1436
44 JUSTIN D SCHILLING 3.0 MI 1199 round6 L 24 L 24 1229
44 JUSTIN D SCHILLING 3.0 MI 1199 round7 W 59 W 59 853
45 DEREK YAN 3.0 MI 1242 round1 L 5 L 5 1655
45 DEREK YAN 3.0 MI 1242 round2 L 51 L 51 1011
45 DEREK YAN 3.0 MI 1242 round3 D 60 D 60 967
45 DEREK YAN 3.0 MI 1242 round4 L 56 L 56 1153

45 DEREK YAN 3.0 MI 1242 round5 W 63 W 63 1175
45 DEREK YAN 3.0 MI 1242 round6 D 55 D 55 1186
45 DEREK YAN 3.0 MI 1242 round7 W 58 W 58 917
46 JACOB ALEXANDER LAVALLEY 3.0 MI 377 round1 W 35 W 35 1438
46 JACOB ALEXANDER LAVALLEY 3.0 MI 377 round2 L 7 L 7 1649
46 JACOB ALEXANDER LAVALLEY 3.0 MI 377 round3 L 27 L 27 1552
46 JACOB ALEXANDER LAVALLEY 3.0 MI 377 round4 L 50 L 50 1056
46 JACOB ALEXANDER LAVALLEY 3.0 MI 377 round5 W 64 W 64 1163
46 JACOB ALEXANDER LAVALLEY 3.0 MI 377 round6 W 43 W 43 1283
46 JACOB ALEXANDER LAVALLEY 3.0 MI 377 round7 L 23 L 23 1363
47 ERIC WRIGHT 2.5 MI 1362 round1 L 18 L 18 1600
47 ERIC WRIGHT 2.5 MI 1362 round2 W 24 W 24 1229
47 ERIC WRIGHT 2.5 MI 1362 round3 L 21 L 21 1563
47 ERIC WRIGHT 2.5 MI 1362 round4 W 61 W 61 955
47 ERIC WRIGHT 2.5 MI 1362 round5 L 8 L 8 1641

47 ERIC WRIGHT 2.5 MI 1362 round6 D 51 D 51 1011
47 ERIC WRIGHT 2.5 MI 1362 round7 L 25 L 25 1745
48 DANIEL KHAIN 2.5 MI 1382 round1 L 17 L 17 1629
48 DANIEL KHAIN 2.5 MI 1382 round2 W 63 W 63 1175
48 DANIEL KHAIN 2.5 MI 1382 round4 D 52 D 52 935
48 DANIEL KHAIN 2.5 MI 1382 round6 L 29 L 29 1602
48 DANIEL KHAIN 2.5 MI 1382 round7 L 35 L 35 1438
49 MICHAEL J MARTIN 2.5 MI 1291 round1 L 26 L 26 1579
49 MICHAEL J MARTIN 2.5 MI 1291 round2 L 20 L 20 1595
49 MICHAEL J MARTIN 2.5 MI 1291 round3 D 63 D 63 1175
49 MICHAEL J MARTIN 2.5 MI 1291 round4 D 64 D 64 1163
49 MICHAEL J MARTIN 2.5 MI 1291 round5 W 58 W 58 917
50 SHIVAM JHA 2.5 MI 1056 round1 L 29 L 29 1602
50 SHIVAM JHA 2.5 MI 1056 round2 W 42 W 42 1332
50 SHIVAM JHA 2.5 MI 1056 round3 L 33 L 33 1449

50 SHIVAM JHA 2.5 MI 1056 round4 W 46 W 46 377
50 SHIVAM JHA 2.5 MI 1056 round6 L 31 L 31 1494
50 SHIVAM JHA 2.5 MI 1056 round7 L 30 L 30 1522
51 TEJAS AYYAGARI 2.5 MI 1011 round1 L 27 L 27 1552
51 TEJAS AYYAGARI 2.5 MI 1011 round2 W 45 W 45 1242
51 TEJAS AYYAGARI 2.5 MI 1011 round3 L 36 L 36 1355
51 TEJAS AYYAGARI 2.5 MI 1011 round4 W 57 W 57 1092
51 TEJAS AYYAGARI 2.5 MI 1011 round5 L 32 L 32 1441
51 TEJAS AYYAGARI 2.5 MI 1011 round6 D 47 D 47 1362
51 TEJAS AYYAGARI 2.5 MI 1011 round7 L 33 L 33 1449
52 ETHAN GUO 2.5 MI 935 round1 W 30 W 30 1522
52 ETHAN GUO 2.5 MI 935 round2 D 22 D 22 1555
52 ETHAN GUO 2.5 MI 935 round3 L 19 L 19 1564
52 ETHAN GUO 2.5 MI 935 round4 D 48 D 48 1382
52 ETHAN GUO 2.5 MI 935 round5 L 29 L 29 1602

52 ETHAN GUO 2.5 MI 935 round6 D 35 D 35
1438
52 ETHAN GUO 2.5 MI 935 round7 L 34 L 34
1399
53 JOSE C YBARRA 2.0 MI 1393 round2 L 25 L
25 1745
53 JOSE C YBARRA 2.0 MI 1393 round4 L 44 L
44 1199
53 JOSE C YBARRA 2.0 MI 1393 round6 W 57
W 57 1092
54 LARRY HODGE 2.0 MI 1270 round1 L 14 L
14 1610
54 LARRY HODGE 2.0 MI 1270 round2 L 39 L
39 1436
54 LARRY HODGE 2.0 MI 1270 round3 L 61 L
61 955
54 LARRY HODGE 2.0 MI 1270 round5 L 15 L
15 1220
54 LARRY HODGE 2.0 MI 1270 round6 L 59 L
59 853
54 LARRY HODGE 2.0 MI 1270 round7 W 64
W 64 1163
55 ALEX KONG 2.0 MI 1186 round1 L 62 L 62
1530
55 ALEX KONG 2.0 MI 1186 round2 D 31 D 31
1494
55 ALEX KONG 2.0 MI 1186 round3 L 10 L 10
1365
55 ALEX KONG 2.0 MI 1186 round4 L 30 L 30
1522

55 ALEX KONG 2.0 MI 1186 round6 D 45 D 45
1242
55 ALEX KONG 2.0 MI 1186 round7 L 43 L 43
1283
56 MARISA RICCI 2.0 MI 1153 round2 L 11 L 11
1712
56 MARISA RICCI 2.0 MI 1153 round3 L 35 L 35
1438
56 MARISA RICCI 2.0 MI 1153 round4 W 45 W
45 1242
56 MARISA RICCI 2.0 MI 1153 round6 L 40 L 40
1348
56 MARISA RICCI 2.0 MI 1153 round7 L 42 L 42
1332
57 MICHAEL LU 2.0 MI 1092 round1 L 7 L 7
1649
57 MICHAEL LU 2.0 MI 1092 round2 L 36 L 36
1355
57 MICHAEL LU 2.0 MI 1092 round3 W 42 W
42 1332
57 MICHAEL LU 2.0 MI 1092 round4 L 51 L 51
1011
57 MICHAEL LU 2.0 MI 1092 round5 L 35 L 35
1438
57 MICHAEL LU 2.0 MI 1092 round6 L 53 L 53
1393
58 VIRAJ MOHILE 2.0 MI 917 round1 W 31 W 31
1494
58 VIRAJ MOHILE 2.0 MI 917 round2 L 2 L 2
1553

58 VIRAJ MOHILE 2.0 MI 917 round3 L 41 L 41 1403
58 VIRAJ MOHILE 2.0 MI 917 round4 L 23 L 23 1363
58 VIRAJ MOHILE 2.0 MI 917 round5 L 49 L 49 1291
58 VIRAJ MOHILE 2.0 MI 917 round7 L 45 L 45 1242
59 SEAN M MC CORMICK 2.0 MI 853 round1 L 41 L 41 1403
59 SEAN M MC CORMICK 2.0 MI 853 round3 L 9 L 9 1411
59 SEAN M MC CORMICK 2.0 MI 853 round4 L 40 L 40 1348
59 SEAN M MC CORMICK 2.0 MI 853 round5 L 43 L 43 1283
59 SEAN M MC CORMICK 2.0 MI 853 round6 W 54 W 54 1270
59 SEAN M MC CORMICK 2.0 MI 853 round7 L 44 L 44 1199
60 JULIA SHEN 1.5 MI 967 round1 L 33 L 33 1449
60 JULIA SHEN 1.5 MI 967 round2 L 34 L 34 1399
60 JULIA SHEN 1.5 MI 967 round3 D 45 D 45 1242
60 JULIA SHEN 1.5 MI 967 round4 D 42 D 42 1332
60 JULIA SHEN 1.5 MI 967 round5 L 24 L 24 1229

61 JEZZEL FARKAS 1.5 ON 955 round1 L 32 L 32 1441
61 JEZZEL FARKAS 1.5 ON 955 round2 L 3 L 3 1384
61 JEZZEL FARKAS 1.5 ON 955 round3 W 54 W 54 1270
61 JEZZEL FARKAS 1.5 ON 955 round4 L 47 L 47 1362
61 JEZZEL FARKAS 1.5 ON 955 round5 D 42 D 42 1332
61 JEZZEL FARKAS 1.5 ON 955 round6 L 30 L 30 1522
61 JEZZEL FARKAS 1.5 ON 955 round7 L 37 L 37 980
62 ASHWIN BALAJI 1.0 MI 1530 round1 W 55 W 55 1186
63 THOMAS JOSEPH HOSMER 1.0 MI 1175 round1 L 2 L 2 1553
63 THOMAS JOSEPH HOSMER 1.0 MI 1175 round2 L 48 L 48 1382
63 THOMAS JOSEPH HOSMER 1.0 MI 1175 round3 D 49 D 49 1291
63 THOMAS JOSEPH HOSMER 1.0 MI 1175 round4 L 43 L 43 1283
63 THOMAS JOSEPH HOSMER 1.0 MI 1175 round5 L 45 L 45 1242
64 BEN LI 1.0 MI 1163 round1 L 22 L 22 1555
64 BEN LI 1.0 MI 1163 round2 D 30 D 30 1522
64 BEN LI 1.0 MI 1163 round3 L 31 L 31 1494
64 BEN LI 1.0 MI 1163 round4 D 49 D 49 1291

64	BEN LI	1.0	MI	1163	round5	L 46	L	46	377
64	BEN LI	1.0	MI	1163	round6	L 42	L	42	1332
64	BEN LI	1.0	MI	1163	round7	L 54	L	54	1270

The following code generates this data:

```
library(tidyverse)
library(gt)

url <-
"https://raw.githubusercontent.com/NafeesKhandker/ELO-
Calculations/refs/heads/main/Chess_result_data.csv"

df <- read_csv(file = url, show_col_types = FALSE, progress
= FALSE)

df |>
  gt()
```

Do the following assignment based on the instructions:
Chess ELO calculations

Based on difference in ratings between the chess players and each of their opponents in our Project 1 tournament, calculate each player's expected score (e.g. 4.3) and the difference from their actual score (e.g 4.0). List the five players who most overperformed relative to their expected score, and the five players that most underperformed relative to their expected score.

You'll find some small differences in different

implementation of ELO formulas. You may use any reasonably-sourced formula, but please cite your source.

Image source (and potentially useful reference!): The Elo Rating System for Chess and Beyond. Feb 15, 2019. [video, 7m]

Great — you have your chess tournament data already in **long format**, which is perfect for analysis. Based on typical assignments (like computing average opponent rating per player), here's how you can proceed in **R using tidyverse**.

✅ 1. Goal (Most Common Requirement)

For each **player**, compute:

- Player name
- State
- Total points
- Pre-rating
- **Average opponent rating**

✅ 2. Load Data

If your data is in CSV:

```
library(tidyverse)
```

R

✅ 3. Clean Data Types

```
df <- df %>%
  mutate(
    pair_num = as.integer(pair_num),
    total_pts = as.numeric(total_pts),
```

R

```
  )
```

✅ 4. Compute Average Opponent Rating

```
df_summary <- df %>%
  group_by(pair_num, player_name, state, total_pts, rating_pre) %>%
  summarise(
```

R

```
  )
```

✅ 5. Format Output (Match Assignment Style)

Most assignments require integer output:

```r
df_summary <- df_summary %>%
```

R

✅ 6. Sort by Player Number

R

✅ 7. Final Output

R

✅ Example Output (for first few players)

pair_num	player_name	state	total_pts	rating_pre	avg_opponent_rating
1	GARY HUA	ON	6.0	1794	1605
2	DAKSHESH DARURI	MI	6.0	1553	1469
3	ADITYA BAJAJ	MI	6.0	1384	1563

✅ 8. Optional: Convert Results (W/L/D → Numeric Score)

If needed:

```
df <- df %>%
  mutate(
    score = case_when(
      result_type == "W" ~ 1,
      result_type == "D" ~ 0.5
```

‹/› R

```
  )
```

✅ 9. Optional: Verify Total Points

```
df_check <- df %>%
  group_by(pair_num, player_name) %>%
```

```
  summarise(
    calculated_pts = sum(score, na.rm = TRUE)
```

R

```
  )
```

🚨 Common Issues You Might Face

❌ Missing rounds (like player 12 or 16)

No problem — `mean(..., na.rm = TRUE)` handles it.

❌ Duplicate rows

Check:

R

❌ Wrong averages

Make sure you are grouping by **player**, not round.

✅ If You Want Next Step

I can help you:

- ✅ Generate **full assignment solution (with plots)**
- ✅ Create **R Markdown (.Rmd) file for submission**
- ✅ Debug your current R code (like your earlier `geom_blank()` error)
- ✅ Build **GitHub-ready project structure**

Just tell me 👍

I want you to apply ELO formula to estimate the expected score from each game data. Then average the expected score that should be compared

Great — now you're moving into **Elo-based expected performance analysis**, which is exactly how chess strength is evaluated.

You want to:

1. Compute **expected score for each game** using the Elo formula
2. Then **average expected score per player**
3. Compare it with actual performance

✅ 1. Elo Expected Score Formula

For each game:

$$E = \frac{1}{1 + 10^{(R_{opp} - R_{player})/400}}$$

Where:

- E = expected score
- R_{player} = player's rating
- R_{opp} = opponent's rating

✅ 2. R Code — Compute Expected Score Per Game

```
library(tidyverse)

df <- df %>%
  mutate(
    rating_pre = as.numeric(rating_pre),
```

‹/› R

```
  )
```

✅ 3. Convert Actual Results to Numeric

```
df <- df %>%
  mutate(
    actual_score = case_when(
```

R

```
    )
  )
```

✅ 4. Aggregate Per Player

Now compute:

- Average expected score
- Actual total score
- Average actual score

```
df_summary <- df %>%
  group_by(pair_num, player_name, state, rating_pre) %>%
  summarise(
    games = n(),
    total_expected = sum(expected_score, na.rm = TRUE),
    avg_expected = mean(expected_score, na.rm = TRUE),

    total_actual = sum(actual_score, na.rm = TRUE),
    avg_actual = mean(actual_score, na.rm = TRUE),
```

R

```
  ) %>%
  arrange(pair_num)
```

✅ 5. Interpretation

Columns:

Column	Meaning
total_expected	Expected total points from Elo
total_actual	Actual points earned
performance_diff	Over/under performance

✅ 6. Example Insight

For **GARY HUA (1794)**:

- Expected score per game ≈ 0.6–0.7
- Over 7 rounds → expected ≈ ~4.2–4.8
- Actual = **6.0**

👉 **He overperformed significantly**

For **ADITYA BAJAJ (1384)**:

- Expected vs stronger players is low (~0.3–0.4)

- Actual = 6.0

👉 **Massive overperformance**

✅ 7. Find Overperformers

```
df_summary %>%
  arrange(desc(performance_diff)) %>%
```

‹/› R

✅ 8. Find Underperformers

```
df_summary %>%
```

‹/› R

✅ 9. Optional: Rating Performance Estimate

You can estimate **performance rating**:

```
df_summary <- df_summary %>%
```

R

✅ 10. Key Insight

- If `performance_diff > 0` → player did better than rating predicts
- If `performance_diff < 0` → player underperformed
- If ≈ 0 → performance matches rating

🚀 If You Want (Next Step)

I can help you:

✅ Generate **full R Markdown report (ready for submission)**
✅ Create **plots (expected vs actual performance)**
✅ Compare **rating vs performance visually**
✅ Add **statistical testing (normal distribution, z-scores)**

Just tell me 👍

break down this code in two parts to show the total expected and then the performance difference

Got it — you want to **separate the logic clearly into two steps**:

1. **Compute total expected score per player**
2. **Compute performance difference (Actual − Expected)**

Here's a clean breakdown 👇

✅ PART 1 — Calculate Expected Score and Total Expected

Step 1: Compute expected score for each game

```
library(tidyverse)

df_expected <- df %>%
  mutate(
    rating_pre = as.numeric(rating_pre),
```

‹/› R

```
  )
```

Step 2: Aggregate total expected score per player

```
expected_summary <- df_expected %>%
  group_by(pair_num, player_name, state, rating_pre) %>%
  summarise(
    games = n(),
    total_expected = sum(expected_score, na.rm = TRUE)
```

‹/› R

```
  )
```

👉 At this stage, you have:

- Total expected points (`total_expected`)
- Average expected per game (`avg_expected`)

✅ PART 2 — Calculate Actual Score & Performance Difference

Step 3: Convert results into numeric score

```
df_actual <- df_expected %>%
  mutate(
    actual_score = case_when(
      result_type == "W" ~ 1,
```

```R
    )
  )
```

Step 4: Aggregate actual score

```R
actual_summary <- df_actual %>%
  group_by(pair_num, player_name) %>%
  summarise(
  )
```

Step 5: Combine and compute performance difference

```R
final_summary <- expected_summary %>%
  left_join(actual_summary, by = c("pair_num", "player_name")) %>%
  mutate(
  arrange(pair_num)
```

✅ Final Output Columns

Column	Meaning
total_expected	Expected score from Elo
total_actual	Actual points scored
performance_diff	Over/under performance

✅ Example Interpretation

- `performance_diff > 0` → player **overperformed**
- `performance_diff < 0` → player **underperformed**

🚀 Optional (for clarity in report)

If your assignment needs cleaner output:

```
final_summary %>%
```

R

If you want, I can next:

- ✅ Add **plots (Expected vs Actual)**
- ✅ Show **top 10 overperformers**
- ✅ Format this into **R Markdown submission format**

breakdown the following code:

```
df_summary <- df %>%
  group_by(pair_num, player_name, state, rating_pre) %>%
  summarise(
    games = n(),
    total_expected = sum(expected_score, na.rm = TRUE),
    avg_expected = mean(expected_score, na.rm = TRUE),

    total_actual = sum(actual_score, na.rm = TRUE),
    avg_actual = mean(actual_score, na.rm = TRUE),

    performance_diff = total_actual - total_expected,
    .groups = "drop"
  ) %>%
  arrange(pair_num)
```

Here is a **clear, step-by-step breakdown** of your code so you understand what every part does 👇

🔷 Full Code

```
df_summary <- df %>%
  group_by(pair_num, player_name, state, rating_pre) %>%
  summarise(
    games = n(),
    total_expected = sum(expected_score, na.rm = TRUE),
    avg_expected = mean(expected_score, na.rm = TRUE),

    total_actual = sum(actual_score, na.rm = TRUE),
    avg_actual = mean(actual_score, na.rm = TRUE),
```

‹/› R

```
  ) %>%
  arrange(pair_num)
```

🧩 Step-by-Step Explanation

🔷 1. `df %>%`

- `%>%` is the **pipe operator** from **dplyr**
- It passes the dataset `df` into the next function

👉 Think: "*Take df and then do this...*"

🔷 2. `group_by(pair_num, player_name, state, rating_pre)`

This groups your data **by player**

Why?

Each player has multiple rows (one per game), so grouping allows us to **summarize per player**

Grouping variables:

- `pair_num` → unique player ID
- `player_name` → player name
- `state` → state
- `rating_pre` → starting rating

👉 After this step:
All rows for the same player are treated as one group

🔷 3. `summarise(...)`

This creates **one row per player** by calculating summary statistics

---

🔶 `games = n()`

- `n()` counts number of rows in each group
- Equals number of games played

👉 Example:
7 rows → 7 games

🔶 `total_expected = sum(expected_score, na.rm = TRUE)`

- Adds all expected scores from Elo formula
- `na.rm = TRUE` ignores missing values

👉 Example:

Code

🔶 `avg_expected = mean(expected_score, na.rm = TRUE)`

- Average expected score per game

👉 Example:

Code

🔶 `total_actual = sum(actual_score, na.rm = TRUE)`

- Adds actual results (W=1, D=0.5, L=0)

👉 Example:

Code

```
1 + 1 + 0 = 2 points
```

🔶 `avg_actual = mean(actual_score, na.rm = TRUE)`

- Average actual score per game

👉 Example:

‹/› **Code**

🔶 `performance_diff = total_actual - total_expected`

This is the **key metric**

👉 Measures how a player performed vs expectation

Value	Meaning
Positive	Player did better than expected
Negative	Player underperformed
Zero	Performed as expected

👉 Example:

```
Actual = 2.0
```

Code

🔶 `.groups = "drop"`

- Removes grouping after summarising
- Returns a normal dataframe

👉 Without this, data stays grouped (which can cause issues later)

🔷 **4.** `%>% arrange(pair_num)`

- Sorts final output by `pair_num`
- Makes results easier to read

📊 Final Output

Each row = **one player**

pair_num	player_name	games	total_expected	total_actual	performance_diff

---

🧠 Concept Summary

You are doing:

👉 Game-level → Player-level aggregation

Metric	Meaning
expected_score	Elo prediction per game
actual_score	Real result

ChatGPT Free offer × Memory full Share •••

total_actual	Actual points
performance_diff	Over/under performance

🚀 Simple Interpretation

- `performance_diff > 0` → strong performance
- `performance_diff < 0` → weak performance
- `performance_diff ≈ 0` → accurate rating

If you want, I can help you **write a short explanation for your assignment report (APA style)** 👍

break down the following code for expected_total and performance_diff"

```
df_summary <- df %>%
  group_by(pair_num, player_name, state, rating_pre) %>%
  summarise(
    games = n(),
    total_expected = sum(expected_score, na.rm = TRUE),
    avg_expected = mean(expected_score, na.rm = TRUE),

    total_actual = sum(actual_score, na.rm = TRUE),
    avg_actual = mean(actual_score, na.rm = TRUE),

    performance_diff = total_actual - total_expected,
    .groups = "drop"
  ) %>%
  arrange(pair_num)
```

Here is a **focused breakdown** of just the two parts you asked about:

👉 `total_expected` and `performance_diff`

🔷 1. `total_expected = sum(expected_score, na.rm = TRUE)`

✅ What it does

This calculates the **total expected score for each player** based on the Elo formula.

🧩 How it works

🔶 `expected_score`

- This is calculated **per game**
- It represents the **probability of winning** (or expected points) against an opponent

👉 Example of Elo expected score:

‹/› R

🔶 `sum(expected_score, na.rm = TRUE)`

- Adds expected scores across all games for that player
- `na.rm = TRUE` means ignore missing values

📊 Example

Game	Expected Score
1	0.75
2	0.60

Game	Expected Score
3	0.40

‹/› R

👉 Interpretation:

- The player was **expected to score 1.75 points** from these games

---

🎯 Meaning

- If a player plays **7 games**, total_expected is the **predicted total points**
- It is based on **rating differences vs opponents**

🔷 2. `performance_diff = total_actual - total_expected`

✅ What it does

This calculates **how much better or worse a player performed compared to expectation**

🧩 How it works

🔶 `total_actual`

- Sum of real results:
  - Win = 1
  - Draw = 0.5
  - Loss = 0

🔶 Subtract expected from actual

```R
```

📊 Example

Metric	Value
total_expected	3.8
total_actual	5.0

```R
```

🎯 Interpretation

Value	Meaning
Positive (+)	Player performed **better than expected**
Negative (−)	Player performed **worse than expected**
0	Player performed **as expected**

📊 More Examples

✅ Overperformance

```
Expected = 4.0
```

R

👉 Player exceeded expectations

❌ Underperformance

```
Expected = 4.0
```

R

👉 Player underperformed

⚖️ Accurate rating

```
Expected = 4.0
```

R

👉 Rating perfectly predicted performance

🧠 Key Idea

- `total_expected` → what Elo predicts
- `total_actual` → what actually happened
- `performance_diff` → difference between them

👉 This is essentially a **player performance evaluation metric**

🚀 Simple Summary

R

If you want, I can also help you **convert this into a clean paragraph for your assignment submission** 👍

I do not want explanation breakdown. I want you to do the code in 2 steps. In the first dataframe, I want to see the column that shows the total_actual and total_expected columns. In the next dataframe I want to see the new column performance_diff:

```
df_summary <- df %>%
  group_by(pair_num, player_name, state, rating_pre) %>%
  summarise(
    games = n(),
    total_expected = sum(expected_score, na.rm = TRUE),
    avg_expected = mean(expected_score, na.rm = TRUE),

    total_actual = sum(actual_score, na.rm = TRUE),
    avg_actual = mean(actual_score, na.rm = TRUE),

    performance_diff = total_actual - total_expected,
    .groups = "drop"
  ) %>%
  arrange(pair_num)
```

Got it — you want the pipeline split into **two separate dataframes**.

✅ Step 1: Create summary with totals (no performance_diff yet)

```
df_totals <- df %>%
  group_by(pair_num, player_name, state, rating_pre) %>%
  summarise(
    games = n(),
    total_expected = sum(expected_score, na.rm = TRUE),
    avg_expected = mean(expected_score, na.rm = TRUE),

    total_actual = sum(actual_score, na.rm = TRUE),
```

‹/› R

```
  ) %>%
  arrange(pair_num)
```

👉 This dataframe will contain:

- `total_expected`
- `total_actual`

✅ Step 2: Add performance difference column

```
df_summary <- df_totals %>%
```

‹/› R

```
    performance_diff = total_actual - total_expected
  )
```

🔥 Result

- `df_totals` → shows totals only
- `df_summary` → includes `performance_diff`

If you want, I can also help you **round values (e.g., 2 decimals) or format output for submission** 👍

I want you to implement the second Elo formula in the calculation and give me the full R code in one shot

Here is the **full R code in one shot**, using both Elo formulas:

- **Expected score formula**
- **Updated rating (second Elo formula)**

This will calculate:

- expected score per game
- actual score

- rating change per game
- final updated rating
- total expected vs actual
- performance difference

✅ Full R Code (with second Elo formula)

```
library(tidyverse)

# Set K-factor (adjust if needed)
K <- 32

df_final <- df %>%
  mutate(
    # Ensure numeric
    rating_pre = as.numeric(rating_pre),
    opponent_rating = as.numeric(opponent_rating),

    # 1. Expected Score (Elo Formula)
    expected_score = 1 / (1 + 10 ^ ((opponent_rating - rating_pre) / 400)),

    # 2. Actual Score
    actual_score = case_when(
      result_type == "W" ~ 1,
      result_type == "D" ~ 0.5,
      result_type == "L" ~ 0
    ),

    # 3. Rating Change (Second Elo Formula)
    rating_change = K * (actual_score - expected_score),
```

```r
    # 4. Updated Rating per game
    updated_rating = rating_pre + rating_change
  )

# ----------------------------------------
# Aggregate per player
# ----------------------------------------

df_summary <- df_final %>%
  group_by(pair_num, player_name, state, rating_pre) %>%
  summarise(
    games = n(),

    # Expected vs Actual
    total_expected = sum(expected_score, na.rm = TRUE),
    total_actual = sum(actual_score, na.rm = TRUE),

    avg_expected = mean(expected_score, na.rm = TRUE),
    avg_actual = mean(actual_score, na.rm = TRUE),
```

‹/› R

```r
    performance_diff = total_actual - total_expected,

    # Rating Changes
    total_rating_change = sum(rating_change, na.rm = TRUE),
    final_rating = first(rating_pre) + total_rating_change,

    .groups = "drop"
  ) %>%
  arrange(pair_num)

# View results
df_summary
```

✅ What this code does

✅ First Elo Formula

Code

→ Expected probability of winning

✅ Second Elo Formula

Code

Where:

- S = actual score (1, 0.5, 0)
- E = expected score
- K = 32 (tunable)

✅ Output Columns

Column	Meaning
total_expected	Expected points

Column	Meaning
total_actual	Actual points
performance_diff	Over/under performance
total_rating_change	Total rating gained/lost
final_rating	New rating after all games

🔥 Optional Improvements

If you want cleaner output:

```
df_summary %>%
  mutate(
    total_expected = round(total_expected, 2),
    avg_expected = round(avg_expected, 3),
```

‹/› R

```
  )
```

If you want, I can next help you:

✅ Show **top overperformers vs underperformers**

✅ Plot **rating changes**

✅ Format into **R Markdown for submission**

List the five players who most overperformed relative to their expected score, and the five players that most underperformed relative to their expected score.

You can easily extract the **top 5 overperformers** and **top 5 underperformers** based on the `performance_diff` column from your `df_summary`. Here's the R code to do that:

```
# Top 5 Overperformers
top_overperformers <- df_summary %>%
  arrange(desc(performance_diff)) %>%
  slice_head(n = 5) %>%
  select(player_name, state, total_actual, total_expected, performance_diff, fina

# Top 5 Underperformers
top_underperformers <- df_summary %>%
  arrange(performance_diff) %>%
  slice_head(n = 5) %>%
```

R

```
# Print results
top_overperformers
top_underperformers
```

- `performance_diff > 0` → player overperformed
- `performance_diff < 0` → player underperformed

This will give you two neat tables with the **players' names, state, total actual & expected points, performance difference, and final rating**.

If you want, I can also make a **single combined table** showing top 5 over- and underperformers side by side for easy comparison. Do you want me to do that?

Give me the 2nd formula in quarto style

Sure! Here's how you can present the **second Elo formula** in **Quarto/R Markdown style** with proper code and inline explanation:

```
## Calculating Elo Ratings (Formula 2)

The second Elo formula updates a player's rating after each game based on the act

\[
R_{\text{new}} = R_{\text{old}} + K \times (S - E)
\]

Where:
```

```
- \(R_{\text{old}}\) = current rating of the player
- \(R_{\text{new}}\) = updated rating after the game
- \(K\) = K-factor (rating adjustment coefficient, e.g., 20, 32)
- \(S\) = actual score of the game (1 = win, 0.5 = draw, 0 = loss)
- \(E\) = expected score of the game calculated as:
\[
E = \frac{1}{1 + 10^{(R_{\text{opponent}} - R_{\text{player}})/400}}
\]

### Example in R

```{r}
# Parameters
K <- 32
```


Markdown

```
df <- df %>%
  mutate(expected_score2 = 1 / (1 + 10^((opponent_rating - rating_pre)/400)))

# Calculate new rating
df <- df %>%
  mutate(updated_rating = rating_pre + K * (actual_score - expected_score2))
```

This computes the **expected score** using the second formula and then updates the player's rating after each game.

```
This format works directly in **Quarto**, showing both the **formula** and a **wo
```

Code
```

give it in the following format:

$$
R_{\text{new}} = R_{\text{old}} + K \times (S - E)
$$

Absolutely! Here's the **second Elo formula** in the exact format you requested for Quarto or Markdown:

```
The second Elo formula updates a player's rating after a game:

$$
R_{\text{new}} = R_{\text{old}} + K \times (S - E)
$$

Where:

- \(R_{\text{old}}\) = current rating of the player
- \(R_{\text{new}}\) = updated rating after the game
- \(K\) = K-factor (adjustment coefficient, e.g., 20, 32)
```

Markdown

```
$$
E = \frac{1}{1 + 10^{(R_{\text{opponent}} - R_{\text{player}})/400}}
$$
```

If you want, I can also **add an R code block** immediately below this in the same Quarto style so that it calculates both `expected_score` and `R_new` for your dataset. Do you want me to do that?

is there anyway to show gt() table in the available width of the screen rather than scrolling right?

Yes! You can make `gt()` **tables** fit the screen width rather than requiring horizontal scrolling by adjusting **column widths** or using the `cols_width()` function in combination with `px()` or `pct()`. Here's how you can do it in R:

Example: Fit `gt()` table to screen width

```
library(gt)

# Example dataframe
df_summary <- data.frame(
  player_name = c("GARY HUA", "DAKSHESH DARURI", "ADITYA BAJAJ"),
  total_actual = c(6, 6, 6),
  total_expected = c(5.8, 5.5, 5.6),
  performance_diff = c(0.2, 0.5, 0.4)
)

# Create gt table
df_summary %>%
```

```
  gt() %>%
  cols_width(
```

R

```
  tab_options(
    table.width = pct(100), # Make table take full screen width
    data_row.padding = px(5) # Optional: reduce padding to save space
  )
```

Key options:

- `cols_width(everything() ~ pct(…))` : Sets column widths as percentages of the table width. You can also specify individual columns, e.g., `cols_width(player_name ~ pct(40), everything() ~ pct(20))` .
- `tab_options(table.width = pct(100))` : Forces the table to take up **100% of the available container width**.
- `data_row.padding = px(…)` : Optional, reduces horizontal spacing to avoid scrolling.

✅ This approach usually prevents horizontal scrolling on most screens, especially in RStudio Viewer or Quarto HTML output.

If you want, I can rewrite your **chess** `gt()` **table code** so it **automatically fits all columns to the screen** for your Elo results. Do you want me to do that?

I tried this:

df_final |>
  gt() %%

cols_width(
everything() ~ pct(33) # Spread columns evenly across the table width
) %>%
tab_options(
table.width = pct(100), # Make table take full screen width
data_row.padding = px(5) # Optional: reduce padding to save space
)
got the following error:
Error in `cols_width()`:
! `data` must either be a `gt_tbl` or a `gt_group`, not a <formula> object.
Run `rlang::last_trace()` to see where the error occurred.

Ah! I see exactly what went wrong — there's a **small syntax issue** in your pipe. You wrote:

```
R
```

Notice the double percent `%%` — that should be a **single pipe-forward** `%>%` if you're using `dplyr` **-style piping**, or keep `|>` but then don't use `%%`. Also, `cols_width()` expects `gt()` **syntax**, not a formula in base R.

Here's the **correct version** using either pipe style:

Using `|>` (R native pipe)

```r
library(gt)

df_final |>
  gt() |>
  cols_width(
    everything() ~ pct(33)  # Spread columns evenly across the table width
  ) |>
```

R

```r
    data_row.padding = px(5)
  )
```

Using `%>%` (magrittr/dplyr pipe)

```r
library(dplyr)
library(gt)

df_final %>%
  gt() %>%
  cols_width(
    everything() ~ pct(33)
  ) %>%
```

R

```r
    data_row.padding = px(5)
  )
```

✅ Key fixes:

1. Use `|>` or `%>%` correctly — don't mix with `%%`.
2. Make sure `library(gt)` is loaded.
3. `everything() ~ pct(33)` works **inside** `cols_width()` **of** `gt()`, not in base R functions.

If you want, I can also give a **version that auto-fits the columns dynamically based on the number of columns**, so you don't have to manually guess the percentages. This usually looks perfect on screen.

Do you want me to do that?

Give me a conclusion in 3/4 lines for this assignment

Here's a concise conclusion you can use:

In this analysis, we calculated each player's expected scores using the Elo rating system and compared them to their actual performance. The results highlighted which players overperformed or underperformed relative to expectations, providing insight into performance consistency. By quantifying performance differences, this approach allows for a fair evaluation of players' relative strengths and weaknesses. Overall, the Elo-based

analysis demonstrates an effective method to predict and assess player outcomes in competitive settings.

I can also make an **even snappier 2–3 line version** suitable for a report summary if you want. Do you want me to do that?

Give me APA style citation for this site: https://www.chess.com/blog/NateSolon/how-elo-ratings-actually-work?

Here's the **APA-style reference** for the blog post you linked (based on the author, title, site, and publication date shown on the page):

Solon, N. (2022, December 12). *How Elo ratings actually work*. Chess.com. https://www.chess.com/blog/NateSolon/how-elo-ratings-actually-work ↗ chess.com

If you need it formatted for a **reference list** or **in-text citation**, let me know!

Sources

Same for https://www.youtube.com/watch?v=AsYfbmp0To0

Here's an **APA-style citation** you can use for the YouTube video you referenced (based on its title, author upload, and posting date from YouTube):

Grime, J. (2019, February 15). *The Elo rating system for chess and beyond* [Video]. YouTube. https://www.youtube.com/watch?v=AsYfbmp0To0 ↗ youtube.com

In-text citation

- Parenthetical: (Grime, 2019)
- Narrative: Grime (2019)

If you need this formatted for a **reference list or footnote**, just let me know!

Sources

GIve me citation for this transcript

Here's an **APA-style citation** for this ChatGPT transcript:

OpenAI. (2026, March 1). *ChatGPT conversation with K. M. Qaiduzzaman on ELO ratings, R analysis, and data visualization* [AI-generated transcript]. OpenAI. https://chat.openai.com/ ↗

In-text citation

- Parenthetical: (OpenAI, 2026)
- Narrative: OpenAI (2026)

This follows APA guidelines for AI-generated content.